

The Otacon Engine

A small, reusable, in-house 2D game engine — architecture and usage manual

Contents

1	What Otacon is	3
1.1	Philosophy	3
2	Building the engine	4
2.1	Compile-time configuration	4
2.2	Source layout	4
3	The four seams	6
3.1	IGame — the game contract	6
3.2	IWindow — window, input, context	6
3.3	IRenderer — drawing	6
3.4	IAudio — sound	7
4	Writing a game	8
5	The drawing API	9
6	The scene system	10
7	Core subsystems	11
7.1	Math and the pluggable scalar	11
7.2	Memory manager	11
7.3	Time	11
7.4	Resources and persistence	11
7.5	Simulation, AI and procedural helpers	12
7.6	Debug runtime	12
8	In-house asset decoders	13
8.1	PNG (asset/Png.cpp, Inflate.cpp)	13
8.2	CAF audio (asset/Caf.cpp)	13
9	Audio	14
10	Graphics backends	15
10.1	The cross-backend contract	15
10.2	Per-backend notes	15
11	Samples	16
11.1	Per-pixel work and the shader seam	16

12 Debug & dev tools	17
13 Verification	18

Chapter 1

What Otacon is

Otacon is a small 2D game engine, built as a static library (`src/engine/`, namespace `otacon`). It is deliberately **game-agnostic**: it knows nothing about any particular game. Three games ship in this repository — *Canabalt* (`src/game/canabalt/`), the Chromium *T-Rex runner* (`src/game/dino/`) and *Flappy Bird* (`src/game/flappy/`) — and each links the same unchanged library, talking to it only through public interfaces. They differ in almost every way a game can: 480×320 / 600×150 / 288×512 logical resolutions, a scrolling follow camera versus a fixed one, a ported generator versus an original one. A fourth game would be one more `IGame` and one more CMake target; nothing in `src/engine/` would move. The golden rule of the layout is: **engine code never mentions the game, and game code never mentions GLFW / OpenGL / Vulkan / CoreAudio**. Everything crosses that boundary through four seams: `IGame`, `IWindow`, `IRenderer` and `IAudio`.

1.1 Philosophy

- **In-house subsystems.** Math (with a fixed-point scalar), an n -dimensional vector library, a memory manager, time management, a debug runtime, the OpenGL function loader, and even the PNG and CAF asset decoders are written from scratch. The only third-party code is the window library (GLFW or SDL2), fetched automatically.
- **Pluggable backends.** The graphics backend (legacy OpenGL, modern OpenGL, Vulkan) and the window backend (GLFW, SDL2) are chosen by a single config flag and selected at compile time — nothing above the platform layer changes.
- **Verifiable.** The renderers are held to a byte-identical contract: every backend draws the same deterministic frame to the same pixels (proven by SHA-256). See Chapter 13.

Chapter 2

Building the engine

The engine is built as part of the project’s CMake build; there is no separate “install”. A game executable links the `otacon` static-library target.

```
cmake -S . -B build          # first configure fetches the window library
cmake --build build -j8      # builds libotacon.a + the game exes
```

2.1 Compile-time configuration

All compile-time choices live in `config/build.cfg`; edit a value and re-run CMake (it reconfigures automatically when the file changes). Any key can also be overridden per-configure, e.g. `-DGRAPHICS_BACKEND=VULKAN`.

Key	Values	Meaning
GRAPHICS_BACKEND	GL_LEGACY / GL_MODERN / VULKAN	Renderer pipeline
WINDOW_BACKEND	GLFW / SDL2	Window / input / context library
SCALAR_TYPE	FLOAT / FIXED	Simulation number type
MEMORY_TRACKING	ON / OFF	Allocation tracking + leak report
DEBUG_RUNTIME	ON / OFF	Runtime debug views

2.2 Source layout

```
src/engine/      # ---- Otacon engine (static lib 'otacon') ----
  core/          #   Timer (scheduling), Tween, Events (Signal + EventBus)
  core/math/     #   Fixed (Q16.16), Scalar (float|fixed), Vector, Rect,
                  #   Random, Noise, Ease
  core/memory/   #   Tracked heap + Arena + Pool allocators
  core/time/     #   Clock + TimeManager (variable/fixed timestep)
  core/debug/    #   DebugRuntime (runtime view switching)
  asset/         #   In-house PNG decode/encode + CAF decode, Resources, SaveData
  audio/         #   IAudio seam, software Mixer, CoreAudio / null backend
  platform/     #   IWindow seam; glfw/ and sdl2/ backends
  render/       #   IRenderer seam; gl_modern/ gl_legacy/ vulkan/; Canvas, Atlas
  scene/        #   Entity, Node, Scene, Camera, Collision, Shapes, Raycast,
                  #   StateMachine, Emitter,
```

```
App.cpp IGame.hpp # TileMap, Animator, Verlet, PathFinder, Steering,
                  # SpatialGrid
src/game/          # ---- games on the engine (canabalt, dino, flappy) ----
src/samples/       # ---- fifteen engine samples (exe 'samples') ----
tests/smoke.cpp    # ---- headless engine self-tests ----
```

Chapter 3

The four seams

3.1 IGame — the game contract

A game is one object implementing `IGame`. The engine's `App` owns the window, renderer, audio device, timing and debug runtime, and drives the game:

```
struct GameContext {           // handed to the game once, at init
    IRenderer*   renderer;     IWindow* window;  IAudio* audio;
    DebugRuntime* debug;       int logicalW, logicalH; const char* assetDir;
};
class IGame {
    virtual void init(GameContext&) = 0;
    virtual void handleInput(const InputFrame&) = 0;    // edges + held state
    virtual void update(Real dt) = 0;                  // one simulation step
    virtual void render(IRenderer&, const DebugRuntime&) = 0;
    virtual void shutdown() {}                          // free GPU/audio handles
    virtual Color clearColor() const; virtual const char* title() const = 0;
};
```

`App::run()` is the main loop: poll input, step the simulation (zero or more fixed steps, or a clamped variable step), then render. The game is handed the renderer to draw with and the `DebugRuntime` so it can honour the clean-play / overlay state.

3.2 IWindow — window, input, context

`createWindow(cfg)` returns the compiled-in backend (GLFW or SDL2). The window exposes a uniform `InputFrame` (level-triggered `held[]` and edge-triggered `pressed[]` arrays keyed by an `Action` enum, plus normalised mouse position and the drag button), the GL context plumbing (`makeContextCurrent` / `swapBuffers` / `glProcAddress`) and the Vulkan plumbing (`vulkanInstanceExtensions` / `createVulkanSurface`). **Both backends are complete** and interchangeable; selecting one is a config flag.

3.3 IRenderer — drawing

See Chapter 5. A backend implements only triangle submission, the texture entry points and the frame lifecycle; all shapes are tessellated once in the shared `IRenderer` base, so every backend

draws identical geometry. Anything beyond that is an *optional capability* a backend may decline — see `supportsShaders()` in Chapter 11.

3.4 IAudio — sound

`createAudio()` returns the platform audio device (CoreAudio on macOS, a silent stub elsewhere); it never returns null, so game code is unchanged when no device is available. The game decodes its sounds in-house and uploads PCM with `createSound`, then fires one-shots with `play` and background music with `playMusic`. See Chapter 9.

Chapter 4

Writing a game

A minimal game is an `IGame` that draws something each frame:

```
class MyGame : public otacon::IGame {
    otacon::IRenderer* r_ = nullptr;
    void init(otacon::GameContext& ctx) override { r_ = ctx.renderer; }
    void handleInput(const otacon::InputFrame& in) override {
        if (in.isPressed(otacon::Action::Jump)) { /* ... */ }
    }
    void update(otacon::Real dt) override { /* advance simulation */ }
    void render(otacon::IRenderer& r, const otacon::DebugRuntime&) override {
        r.fillRect(100, 100, 64, 64, otacon::Color::rgb(0xf0c020));
    }
    const char* title() const override { return "My Game"; }
};
```

`main()` creates the `App`, the game, and runs:

```
otacon::App app;  MyGame game;
if (app.init(cfg, &game, assetDir)) app.run();
app.shutdown();
```

Chapter 5

The drawing API

The renderer works in a fixed **logical** coordinate space — 480×320 by default, origin top-left, y downward — and each backend applies its own projection to clip space. Colours are straight-alpha RGBA. The public primitives:

Call	Draws
<code>fillRect(x,y,w,h,color)</code>	a filled axis-aligned rectangle
<code>fillRotatedRect(cx,cy,w,h,deg,color)</code>	a rectangle rotated about its centre
<code>drawRectOutline(x,y,w,h,color,thick)</code>	a rectangle outline
<code>drawLine(x0,y0,x1,y1,color,thick)</code>	a thick line segment
<code>drawImage(tex,x,y,w,h,u0,v0,u1,v1,tint)</code>	a texture sub-rect into a dest rect
<code>drawImageRotated(tex,cx,cy,w,h,deg,uv,tint)</code>	a rotated textured quad
<code>drawText(text,x,y,scale,color)</code>	a string in the built-in 3×5 bitmap font

Textures are created from a decoded `Image` with `createTexture(img, repeat)` (nearest filtering, for crisp pixel art) and freed with `destroyTexture`. `updateTexture(tex, w, h, rgba)` replaces an existing texture's pixels at the same extent: it is the upload path for anything rasterised on the CPU each frame, and re-using the texture object avoids churning a GPU allocation every frame.

The cross-backend contract: a backend implements only `submitTriangles` (solid) and `submitTextured`, the texture entry points, and `beginFrame/endFrame`. Every shape above — rectangles, oriented lines, outlines, the bitmap font — is tessellated to triangles once, in the shared base class, so all backends rasterise byte-for-byte identical geometry.

Chapter 6

The scene system

The engine offers a small scene graph so rendering lives in one place — “the Scene renders the tree, nothing draws on the side.”

- **Entity** — simulation / collision data: position, size, velocity, acceleration, drag, the **solid/fixed/moves** flags, a **scrollFactor** for parallax, and an optional texture/colour for the entity draw layer. **updateMotion** integrates it.
- **Node** — a scene-tree element with **update(dt, cam)** and **render(r, cam)**; concrete nodes are particle emitters, sprite drawers, facade renderers, UI. A node may also override **renderColliders** (debug) and is skipped when **!visible**.
- **Scene** — owns the camera, an ordered entity draw layer (rebuilt each frame by the game), a backdrop node list (drawn *before* the entities, for the farthest parallax), and the main node list (drawn after). It ticks and renders the whole tree, then the debug overlays.
- **Camera** — a faithful flixel follow camera: centres a target, lerps the scroll, clamps to bounds, and projects world to screen through each entity’s **scrollFactor** (**screenPoint**). Also hosts the decaying screen-quake.
- **Collision** — flixel’s swept-AABB separation (**solveX** then **solveY**) with the same direction bookkeeping and extent rejection that avoids tunnelling.
- **Emitter** — a particle emitter (a **Node**) with a fixed pool, burst and continuous modes, gravity/drag/spin, and an optional sprite sheet so particles can be solid quads or textured frames.

Chapter 7

Core subsystems

7.1 Math and the pluggable scalar

Float is the default and fixed-point is a didactic option, not a design commitment the engine is built around. Choose `FIXED` when you want what it actually offers — bit-identical results across machines, for lockstep netcode or an exactly reproducible replay — and remember that `Q16.16` tops out at ± 32767.99998 , which an endless runner’s world `X` exceeds. Building the test suite both ways does make a useful second opinion on floating-point assumptions, but that is a bonus rather than the reason the option exists.

`Real` is the engine number type. With `SCALAR_TYPE=FLOAT` it is an IEEE float (and reproduces the reference game bit-for-bit). With `FIXED` it becomes `Fixed`, an in-house *Q16.16* fixed-point type — a performance/determinism teaching point. (`Q16.16` saturates near ± 32768 , so an endless runner’s world-*x* eventually overflows; that is itself the lesson about range.) Vectors (`Vector.cpp`) are templated on the scalar and explicitly instantiated for 2, 3 and 4 dimensions.

7.2 Memory manager

`core/memory` provides three classic allocators: a **tracked heap** (records every live block with size/tag/call-site and prints leaks at exit), a linear **Arena** (bump-pointer, reset per frame) and a fixed-size **Pool** (free-list, $O(1)$, zero fragmentation). Global `new/delete` are intentionally *not* overridden, so third-party allocations stay separate.

7.3 Time

`TimeManager` turns wall-clock time into the per-frame `dt`. The *variable* policy reproduces fixel exactly (`dt = now - last`, clamped to `maxElapsed = 1/20 s`); the *fixed* policy banks real time and emits whole fixed steps for frame-rate-independent determinism. It also drives the time scale used by the slow-mo / fast-forward debug control.

7.4 Resources and persistence

`Resources` is an engine-owned, load-once cache for textures and sounds, handed to every game through `GameContext`. A single owner is the point: two systems asking for the same file get the same handle, and everything is released once, after the game shuts down and while the renderer is

still alive. Before it existed the three games carried 23 hand-written `loadPng` calls and 34 matching `destroyTexture` calls between them; there are now none. `SaveData` is the matching store for small persistent state — a high score, a settings flag — writing atomically and reading totally.

7.5 Simulation, AI and procedural helpers

Beyond `Entity` and the AABB solver, the scene layer carries the subsystems a 2D game reaches for: `SpatialGrid` (a uniform-grid broad phase, so collision need not be $O(n^2)$), `Raycast` (ray/AABB by the slab method, ray/tilemap by a DDA walk, and the line-of-sight query they stand in for), `Verlet` (position-based dynamics for ropes, bridges and cloth), `PathFinder` (A* over a `TileMap`, with the heuristic exposed as the knob it really is), `Steering` (flocking plus seek/flee/arrive), `TileMap` and `Animator`. `core/math` adds `Random`, `Noise` and `Ease`.

`Random` is worth singling out. Before it existed, fifteen separate places in this project had each written their own linear congruential generator, two of them inside the engine. A generator here is a value rather than a global, so two systems can neither perturb each other's stream nor be made irreproducible by the order they happen to run in — which is why the camera's quake keeps a private one, a cosmetic shake having no business shifting the stream a level generator reads.

7.6 Debug runtime

`DebugRuntime` holds independently toggleable views (colliders, camera focus, parallax bands, grid, wireframe, velocities, HUD) plus pause/step and curated presets. Because the views are plain state read by the engine each frame, every backend gets identical debug visualisation for free. A single “UI hidden” flag gives clean-play (and is what `--capture` uses for deterministic frames).

Chapter 8

In-house asset decoders

8.1 PNG (`asset/Png.cpp`, `Inflate.cpp`)

Rather than link an image library, the engine decodes PNGs itself. `Inflate.cpp` is a from-scratch DEFLATE decompressor (RFC 1951) with its own bit reader, canonical Huffman decoder and LZ77 window, wrapped for zlib streams (RFC 1950). `Png.cpp` parses the chunk list (IHDR/PLTE/tRNS/IDAT/IEND), inflates, undoes the per-scanline filters (None/Sub/Up/Average/Paeth) and expands grayscale / RGB / palette+tRNS / RGBA into straight RGBA8. `PngWrite.cpp` is a matching encoder (valid PNG via stored/uncompressed DEFLATE blocks — no compressor needed) used for screenshots and the cross-backend diff.

8.2 CAF audio (`asset/Caf.cpp`)

`loadCaf` parses the (big-endian) Core Audio Format container — the `desc` chunk for the format, the `data` chunk for the samples — and decodes LPCM of any width/endianness into interleaved float in $[-1, 1]$ as an `AudioClip`. (Sounds in the reference game ship as 22050 Hz mono 16-bit LPCM.)

Chapter 9

Audio

The **IAudio** device owns an in-house software **Mixer** and a thin platform output. The mixer holds the uploaded sounds and a pool of one-shot voices plus one looping music voice, sums them into an interleaved stereo float buffer, applies master + per-voice gain and a soft clamp, and resamples on upload so the real-time render path never converts rates. A short mutex guards the voice list between the game thread (**play**) and the audio thread (**render**). The **CoreAudio** backend drives a `DefaultOutput AudioUnit` whose render callback pulls from the mixer; the **null** backend (non-Apple) accepts everything and stays silent. This is the only OS-specific audio code — exactly as GLFW is the only OS-specific window code.

Chapter 10

Graphics backends

10.1 The cross-backend contract

Every backend implements only triangle rasterisation plus the frame lifecycle. The contract:

- **Coordinates:** logical 480×320 , origin top-left, y down; each backend applies its own orthographic projection.
- **Blending:** straight alpha, `SRC_ALPHA / ONE_MINUS_SRC_ALPHA`.
- **Primitive:** triangle lists only; no backend-specific shapes.
- **Capabilities:** anything a backend cannot do is a question it is asked, never an assumption. `supportsShaders()` is the only one so far, and whatever is built on it must also work without it, so no screen becomes reachable on some backends and not others.

10.2 Per-backend notes

- **GL Legacy (2.1).** `glOrtho` + immediate-mode `glBegin/glColor/glVertex`; a compatibility context, no loader needed. Fixed-function throughout, so there is no programmable stage at all and `supportsShaders()` is false.
- **GL Modern (core 3.3).** A streamed VBO/VAO and a small vertex/fragment shader pair (the vertex shader does the logical→clip projection); needs a core, forward-compatible context and a function loader — shipped in-house in `gl_modern/GLLoader` (no GLAD/GLEW). The only backend with a reachable programmable stage, so it is the one where `supportsShaders()` is true and user fragment effects compile.
- **Vulkan.** A full implementation: instance (MoltenVK portability), device, swapchain, render pass, solid + textured pipelines from embedded SPIR-V, per-frame dynamic vertex buffers, staging-buffer texture uploads, and per-frame/per-image synchronisation. Key difference, handled in the shader: Vulkan clip space already has y pointing down, so it does *not* flip y as the GL shaders do. Validation-clean. Shaders are pre-compiled to SPIR-V and embedded at build time, so there is no runtime compiler and `supportsShaders()` is false; `updateTexture` is supported, via a staging buffer and a one-time command per upload.

Vulkan on macOS uses MoltenVK via the Homebrew loader; launch through `tools/run.sh` (it sets `DYLD_FALLBACK_LIBRARY_PATH` and `VK_ICD_FILENAMES`). It takes the executable to launch — `canabalt` (the default), `dino`, `flappy` or `samples` — so `tools/run.sh samples` is how the sample gallery is run on this backend.

Chapter 11

Samples

Twenty sample screens (`src/samples/`, executable `samples`) exercise the engine on its own. The first fifteen are the canonical set; the rest cover the facilities those do not reach, on the rule that no subsystem ships without something that exercises it — a facility nothing calls is indistinguishable from one that does not work. They link `otacon` and nothing else, and generate every texture they use in code — a sample that borrowed a game’s art would mean a seam had leaked. One gallery object hosts them all, so adding a sample is one `.cpp` plus one row in `Registry.cpp`.

11.1 Per-pixel work and the shader seam

Two samples (2D lighting, and the shader playground) need per-fragment control, which sits awkwardly against the cross-backend contract: only the modern OpenGL backend has a reachable programmable stage. The engine therefore offers both halves and lets a sample choose:

- `updateTexture(tex, w, h, rgba)` re-uploads an existing texture’s pixels. A sample shades a buffer in plain C++ and pushes it once per frame. This works on every backend, GL legacy included.
- `createEffect(fragmentSrc, log, logSize)` compiles a user fragment shader, with `useEffect` and `setEffectUniform` to drive it. The engine keeps the vertex stage so the projection stays identical. `supportsShaders()` reports whether the backend can do this at all: true for GL modern, false for GL legacy (fixed-function) and for Vulkan (pre-built SPIR-V, no runtime compiler).

Every sample that uses an effect also carries the CPU path, so no sample is backend-exclusive and Chapter 13’s contract still holds everywhere.

Chapter 12

Debug & dev tools

These are engine-level and work in any game (state shown in the HUD when visible):

- **Perf + memory overlay (H)**. Frame-time graph (60/30 fps lines), live draw-call/vertex counts read from the renderer, and the memory manager's live/peak bytes — all backend-agnostic.
- **Time scale (+/-)**. Scales the *input* time $0.1\times..4\times$, so the fixed-timestep `dt` is unchanged and slow-mo keeps determinism.
- **Screenshots (F12) + capture**. The renderer reads the framebuffer back and writes it through the in-house PNG encoder. `--capture <png> --frames N` renders N deterministic frames (UI hidden) and quits — used by `tools/backend-diff.sh`.
- **Clean play (`)**. Hides all dev UI at once.

Chapter 13

Verification

The renderers are held to the byte-identical contract by `tools/backend-diff.sh`, which renders the same deterministic frame in each graphics backend and compares the PNGs. With both window backends in play, the full matrix — $\{\text{GLFW, SDL2}\} \times \{\text{GL_MODERN, GL_LEGACY, VULKAN}\}$ — produces **the same SHA-256 across all six combinations**: the rigorous proof that neither the window seam nor the graphics seam perturbs a single pixel. The headless `otacon_smoke` target additionally unit-tests the integrator, collision, the emitter, the fixed-point math, the PNG/-CAF decoders and the audio mixer.